

Praktikum 6 — Regression

Data Science · Magister Ilmu Komputer, FMIPA UGM Companion to **Chapter 6**. Assignment weight: **3%**.

What you will do

1. Derive the OLS estimator three ways and confirm they agree.
2. Choose and read regression metrics — including the two that R^2 hides.
3. Diagnose the Gauss–Markov assumptions on real residuals, and find the one point that moves a whole line.
4. Watch multicollinearity destroy coefficients while predictions stay fine.
5. Compare ridge, lasso and elastic net, and choose λ from a CV curve.
6. Fit logistic regression and read coefficients as **odds ratios**.
7. Choose a decision threshold from **costs**, not from the default 0.5.

The datasets. `data/rumah_yogya.csv` (2,200 house listings) and `data/credit.csv`.

How to read this notebook

Every code cell is followed by a short explanation in a box like this one:

What just happened. What the code did, in plain words.

How to read the output. Which numbers to look at, and what they mean.

Why it matters. The one idea to take away.

Run a cell, look at its output, *then* read the box underneath.

Four ideas are used throughout, so learn them once here.

- **RMSE** is the typical size of an error, in the same units as the target (here, millions of rupiah). **MAE** is the same idea using absolute rather than squared errors, so it is less disturbed by a few large misses. Lower is better for both.
- **R^2** is the share of the target's variance the model explains: 1 is perfect, 0 is no better than always predicting the mean, and on a test set it can even go *negative*.
- **Train/test split.** We fit on one part of the data and report scores on a part the model has never seen. A score on the training rows measures memory, not skill.
- **Residual = actual – predicted.** Almost every diagnostic in this notebook is a way of looking at residuals. A good model leaves residuals that look like formless noise; any *pattern* left in them is signal the model failed to use.

```

import sys
from pathlib import Path

ROOT = Path.cwd()
while not (ROOT / "checks.py").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent
sys.path.insert(0, str(ROOT))

import checks
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt

DATA = ROOT / "data"
plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False,
                    "axes.spines.right": False, "axes.grid": True,
                    "grid.alpha": 0.3, "axes.titleweight": "bold"})
pd.set_option("display.width", 110)

```

```

rumah = pd.read_csv(DATA / "rumah_yogya.csv")
print(rumah.shape)
print(rumah.describe().T.round(1).to_string())
rumah.head()

```

What just happened. We loaded 2,200 Yogyakarta house listings. `harga` (price, in millions of rupiah) is the target; the rest are candidate predictors.

How to read the output. Compare the median (50%) with the mean of `harga`: the mean is well above the median, and the maximum is several times either. That is a **right-skewed** price distribution, and it is the reason §2 ends up modelling `log(harga)` rather than `harga`.

1. Ordinary least squares, three ways (DEMO)

OLS minimises $\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2$. Differentiating and setting to zero gives the **normal equations**, hence

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^{\top} \mathbf{X})^{-1} \mathbf{X}^{\top} \mathbf{y}$$

Let us confirm that the closed form, a numerical solver, and `scikit-learn` all agree — and then see why nobody uses the closed form in practice.

```

from sklearn.linear_model import LinearRegression

X_simple = rumah[["luas_bangunan"]].to_numpy()
y_simple = rumah["harga"].to_numpy()

# --- route 1: the closed form, by hand -----
Xd = np.column_stack([np.ones(len(X_simple)), X_simple])
beta_closed = np.linalg.inv(Xd.T @ Xd) @ Xd.T @ y_simple

# --- route 2: the scalar formula, slope = Cov(x,y)/Var(x) -----
x = X_simple.ravel()
slope = np.cov(x, y_simple, ddof=1)[0, 1] / np.var(x, ddof=1)
intercept = y_simple.mean() - slope * x.mean()

# --- route 3: scikit-learn (which uses the SVD, not the inverse) -----
lr = LinearRegression().fit(X_simple, y_simple)

print(f"closed form : intercept {beta_closed[0]:9.4f}    slope {beta_closed[1]:.6f}")
print(f"Cov/Var      : intercept {intercept:9.4f}    slope {slope:.6f}")
print(f"scikit-learn: intercept {lr.intercept_:9.4f}    slope {lr.coef_[0]:.6f}")

```

What just happened. Three different routes to the same straight line through (building area, price): the matrix formula, the one-predictor formula, and the library.

How to read the output. All three agree to six decimals. The slope says each extra square metre of building is worth about 10 million rupiah; the intercept is the price of a house with zero area — arithmetic, not meaning.

Why it matters. `LinearRegression` is not a black box. It is this formula, and you have just checked it by hand.

The slope also equals $r \cdot s_y / s_x$ — the correlation scaled by the ratio of standard deviations. That identity is why **regression to the mean** happens: since $|r| \leq 1$, a value one SD above the mean in x predicts *fewer* than one SD above the mean in y .

```

r = np.corrcoef(x, y_simple)[0, 1]
print(f"r * s_y/s_x = {r * y_simple.std(ddof=1) / x.std(ddof=1):.6f}")
print(f"fitted slope = {slope:.6f}")
print(f"ncorrelation r = {r:.4f}: a house 1 SD above average in size is")
print(f"predicted at only {r:.2f} SD above average in price.")

```

What just happened. We recomputed the same slope from the correlation and the two standard deviations.

How to read the output. Identical to the fitted slope. And since $r \approx 0.83$, a house one standard deviation larger than average is predicted only 0.83 standard deviations more expensive — not one.

Why it matters. This shrinkage towards the mean is built into *every* least-squares prediction. It is not a flaw, and it is why extreme cases are predicted less extremely than they turn out to be.

```

# Two properties worth verifying: residuals sum to zero, and the line
# passes through the centroid.
resid = y_simple - lr.predict(X_simple)
print(f"sum of residuals      : {resid.sum():.6e} (should be ~0)")
print(f"line at mean(x)         : {lr.predict([[x.mean()]])[0]:.4f}")
print(f"mean(y)                   : {y_simple.mean():.4f}")

```

What just happened. Two algebraic consequences of including an intercept, checked numerically.

How to read the output. The residuals sum to about $1e-11$ — zero, up to floating-point noise — and the fitted line passes exactly through the point (mean x , mean y).

Why it matters. Both follow from the normal equations, so they hold for *every* OLS fit with an intercept. If your residuals do not sum to zero, you have a bug, not a discovery.

Nobody computes $(X^T X)^{-1}$ in practice. Forming the inverse squares the condition number and loses precision. Numerical libraries solve the least squares problem by QR or SVD instead. The closed form is for understanding.

```
# Demonstrating the numerical hazard on a near-singular design.
rng = np.random.default_rng(0)
a = rng.normal(size=200)
Xbad = np.column_stack([np.ones(200), a, a + rng.normal(0, 1e-7, 200)])
ybad = 3 + 2 * a + rng.normal(0, 0.1, 200)

cond = np.linalg.cond(Xbad.T @ Xbad)
print(f"condition number of X'X : {cond:.3e}")
try:
    b_inv = np.linalg.inv(Xbad.T @ Xbad) @ Xbad.T @ ybad
    print("via explicit inverse      :", np.round(b_inv, 4))
except np.linalg.LinAlgError as e:
    print("via explicit inverse      : FAILED —", e)
b_lstsq, *_ = np.linalg.lstsq(Xbad, ybad, rcond=None)
print("via lstsq (SVD)              :", np.round(b_lstsq, 4))
```

What just happened. We built a design matrix with two almost identical columns (they differ by $1e-7$) and solved it both ways.

How to read the output. The condition number is about $1e14$ — enormous. Both routes recover the intercept of 3, but the two coefficients on the near-identical columns are wild: around $\pm 60,000$, and *different* between the two methods. Their **sum** is still about 2, the truth.

Why it matters. When columns are nearly duplicated, the individual coefficients are numerically meaningless while the prediction stays fine. That is the numerical face of multicollinearity, which §3 revisits as a statistical problem.

2. Multiple regression, metrics and the assumptions (DEMO)

The Gauss–Markov theorem: under assumptions 1–4, OLS is the **Best Linear Unbiased Estimator**. Assumption 5 (normality) is needed only for small-sample inference — *not* for the estimator to be BLUE.

Note what is **not** assumed: normality of y , or of the predictors.

```

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

feat_num = ["luas_bangunan", "luas_tanah", "kamar_tidur", "kamar_mandi",
            "umur_bangunan", "jarak_pusat_km"]
X = pd.get_dummies(rumah[feat_num + ["kecamatan", "sertifikat"]],
                  drop_first=True).astype(float)
yv = rumah["harga"]

Xtr, Xte, ytr, yte = train_test_split(X, yv, test_size=0.2, random_state=42)
model = LinearRegression().fit(Xtr, ytr)
pred = model.predict(Xte)

n_te, p_pred = len(yte), X.shape[1]
r2 = r2_score(yte, pred)
adj_r2 = 1 - (1 - r2) * (n_te - 1) / (n_te - p_pred - 1)
mape = np.mean(np.abs((yte - pred) / yte)) * 100

print(f"held-out RMSE      : {mean_squared_error(yte, pred) ** 0.5:8.2f} juta")
print(f"held-out MAE       : {mean_absolute_error(yte, pred):8.2f} juta")
print(f"held-out MAPE       : {mape:8.2f} %")
print(f"held-out R^2        : {r2:8.4f}")
print(f"held-out adj. R^2   : {adj_r2:8.4f}   ({p_pred} predictors, {n_te} test rows)")

```

What just happened. We fitted a model on 80% of the houses, predicted the other 20%, and reported five metrics on those unseen rows.

How to read the output. RMSE about 224 and MAE about 162 million rupiah, against a median house price of 800 million — so typical errors are roughly a fifth of the price, which MAPE states directly as about 19%. R^2 of 0.79 sounds impressive; "wrong by 224 million rupiah" sounds less so. Both describe the same model.

RMSE is larger than MAE, always, and the size of the gap tells you how uneven the errors are: a few large misses pull RMSE up and barely move MAE.

Adjusted R^2 charges a fee for every predictor, so unlike R^2 it can go *down* when you add a useless column. That is why it, and not R^2 , can be used to compare models of different sizes.

Why it matters. Always quote an error in the units of the target next to any R^2 . One of the two is a statement a house buyer could act on.

Why RMSE and MAE part company (short demo)

Take 200 ordinary errors and push **one** of them further and further out.

```

rng_m = np.random.default_rng(93)
err = rng_m.normal(0, 1.0, 200)

rows = []
for shift in (0, 5, 10, 25):
    e = err.copy()
    e[0] = shift                    # one error, made progressively worse
    rows.append({"outlier_error": shift,
                "RMSE": np.sqrt(np.mean(e ** 2)),
                "MAE": np.mean(np.abs(e))})
print(pd.DataFrame(rows).round(3).to_string(index=False))

```

What just happened. 199 errors stayed exactly the same; one grew from 0 to 25.

How to read the output. MAE crawls from 0.78 to 0.90, because one point in 200 can only move an average of absolute values so far. RMSE doubles, from 0.98 to 2.02, because squaring makes that single error dominate the sum.

Why it matters. Report both. If RMSE is much larger than MAE, your errors are concentrated in a few bad cases — go and look at them. If they are close, the errors are evenly spread. Which metric you *optimise* is a decision about whether one 25-unit miss is worse than twenty-five 1-unit misses.

Residual diagnostics

The residuals-against-fitted plot is the single most informative diagnostic. Under a correct model it shows a formless horizontal band.

```
fitted = model.predict(Xtr)
res = ytr - fitted
std_res = res / res.std()

fig, axes = plt.subplots(1, 3, figsize=(14, 3.6))

axes[0].scatter(fitted, res, s=8, alpha=0.35, color="#2E6F9E")
axes[0].axhline(0, color="#9C3535", lw=1)
axes[0].set(title="Residuals vs fitted", xlabel="fitted (juta)", ylabel="residual")

stats.probplot(std_res, dist="norm", plot=axes[1])
axes[1].set_title("Normal Q-Q")
axes[1].get_lines()[0].set(markersize=3, alpha=0.4, color="#2E6F9E")
axes[1].get_lines()[1].set(color="#9C3535")

axes[2].scatter(fitted, np.sqrt(np.abs(std_res)), s=8, alpha=0.35,
               color="#2E6F9E")
axes[2].set(title="Scale-location", xlabel="fitted",
            ylabel=r"$\sqrt{|\text{standardised residual}|}$")
plt.show()
```

How to read the figure. Three standard diagnostics.

- **Residuals vs fitted** (left) should be a formless horizontal band. It is not: it is a *cone*, narrow at cheap houses and wide at expensive ones.
- **Normal Q–Q** (middle) compares the residuals with a normal distribution. Points on the red line mean normal; the ends curve away, so the residuals have heavier tails than normal.
- **Scale-location** (right) plots the size of the residuals against the fitted value. A rising trend is the same cone stated as a line.

Why it matters. The cone is **heteroscedasticity**: the error variance grows with the fitted value. Nothing in RMSE or R^2 would have told you.

The funnel shape in panels 1 and 3 is **heteroscedasticity**: the error variance grows with the fitted value. That is exactly what you expect when the true relationship is multiplicative and you fit it additively.

Formal test, without `statsmodels` :

```
def breusch_pagan(residuals, X_design):
    """LM test for heteroscedasticity: regress squared residuals on X."""
    e2 = residuals ** 2
    Xd = np.column_stack([np.ones(len(X_design)), np.asarray(X_design, float)])
    beta, *_ = np.linalg.lstsq(Xd, e2, rcond=None)
    ss_res = ((e2 - Xd @ beta) ** 2).sum()
    ss_tot = ((e2 - e2.mean()) ** 2).sum()
    r2 = 1 - ss_res / ss_tot
    lm = len(e2) * r2
    return lm, 1 - stats.chi2.cdf(lm, Xd.shape[1] - 1)

lm, p_bp = breusch_pagan(res.to_numpy(), Xtr)
print(f"Breusch-Pagan LM = {lm:.2f}, p = {p_bp:.3e}")
print("-> heteroscedasticity confirmed. Coefficients remain UNBIASED;")
print("    only the standard errors are wrong.")
```

What just happened. The Breusch–Pagan test regresses the *squared* residuals on the predictors. If the size of the error depends on the inputs, that regression finds it.

How to read the output. p is essentially 0, so "the error variance is constant" is rejected outright. The eye and the test agree.

Why it matters. Heteroscedasticity does **not** bias the coefficients — they still estimate the right thing on average. What breaks is every standard error, t -statistic, p -value and confidence interval the model reports. If you only predict, you can live with it; if you make claims, you cannot.

One point can own the line: leverage and Cook's distance (short demo)

Leverage measures how unusual a row's *predictors* are; **Cook's distance** measures how much the fitted line would move if that row were deleted. Here, 24 well-behaved points plus one observation far out in x .

```

rng_1 = np.random.default_rng(2)
x_1 = np.sort(rng_1.uniform(1, 6, 24))
y_1 = 2 + 1.1 * x_1 + rng_1.normal(0, 0.6, 24)
x_all = np.append(x_1, 14.0)          # far out in x ...
y_all = np.append(y_1, 5.0)          # ... and off the line

slope_with = np.polyfit(x_all, y_all, 1)[0]
slope_without = np.polyfit(x_1, y_1, 1)[0]

D = np.column_stack([np.ones(len(x_all)), x_all])
H = D @ np.linalg.inv(D.T @ D) @ D.T      # the "hat" matrix
h = np.diag(H)                             # leverage of each row
beta_all, *_ = np.linalg.lstsq(D, y_all, rcond=None)
e = y_all - D @ beta_all
s2 = (e ** 2).sum() / (len(y_all) - 2)
cook = e ** 2 / (2 * s2) * h / (1 - h) ** 2

print(f"slope with the point      : {slope_with:.3f}")
print(f"slope without it         : {slope_without:.3f}")
print(f"\nleverage of that point : {h[-1]:.3f} (rule of thumb: investigate above "
      f"{2 * 2 / len(x_all):.2f})")
print(f"Cook's distance           : {cook[-1]:.1f} (rule of thumb: investigate above 1)")
print(f"largest Cook's D among the other 24: {np.sort(cook)[-2]:.3f}")

fig, ax = plt.subplots(figsize=(6, 3.4))
ax.scatter(x_1, y_1, s=18, color="#2E6F9E", label="24 ordinary points")
ax.scatter([14.0], [5.0], s=60, color="#9C3535", label="one odd point")
gx = np.array([0.0, 15.0])
ax.plot(gx, np.polyval(np.polyfit(x_all, y_all, 1), gx), color="#9C3535",
      label=f"with it (slope {slope_with:.2f})")
ax.plot(gx, np.polyval(np.polyfit(x_1, y_1, 1), gx), color="#356B3F", ls="--",
      label=f"without it (slope {slope_without:.2f})")
ax.set(xlabel="x", ylabel="y", title="One point, and the slope it steals")
ax.legend(fontsize=8, frameon=False)
plt.show()

```

What just happened. We fitted the same 25 points twice — once with the odd point, once without — and measured that point's leverage and Cook's distance.

How to read the output. The slope collapses from **1.165** to **0.260**: the single point has turned a clear rising relationship into an almost flat one. Its leverage is 0.74 against a rule-of-thumb threshold of 0.16, and its Cook's distance is about 27 where anything above 1 deserves attention — while no other point exceeds 0.15.

Leverage versus influence. Leverage is *potential*: being far out in x. A high-leverage point that sits on the line changes nothing. Influence (Cook's distance) is leverage **and** being off the line, and that is what moves the fit.

Why it matters. Never delete such a point automatically. It is either a data error (fix it), a different population (model it separately), or the most informative observation you have (keep it, and say that the conclusion rests on it).

The fix: model on the log scale

Chapter 3 showed price is log-normal and its relationship to area is a power law. Modelling $\log(\text{harga})$ against $\log(\text{luas})$ makes the relationship linear *and* the errors homoscedastic.

```

Xlog = X.copy()
Xlog["luas_bangunan"] = np.log(Xlog["luas_bangunan"])
Xlog["luas_tanah"] = np.log(Xlog["luas_tanah"])
ylog = np.log(yv)

Xtr_1, Xte_1, ytr_1, yte_1 = train_test_split(Xlog, ylog, test_size=0.2,
                                             random_state=42)
model_log = LinearRegression().fit(Xtr_1, ytr_1)

res_log = ytr_1 - model_log.predict(Xtr_1)
lm2, p_bp2 = breusch_pagan(res_log.to_numpy(), Xtr_1)

print(f"R^2 on the log scale      : {model_log.score(Xte_1, yte_1):.4f}")
print(f"Breusch-Pagan p, raw      : {p_bp:.3e}")
print(f"Breusch-Pagan p, log      : {p_bp2:.3e}")

fig, axes = plt.subplots(1, 2, figsize=(10, 3.4))
axes[0].scatter(model.predict(Xtr), res, s=7, alpha=0.3, color="#9C3535")
axes[0].axhline(0, color="black", lw=0.8)
axes[0].set(title="Raw scale: funnel", xlabel="fitted", ylabel="residual")
axes[1].scatter(model_log.predict(Xtr_1), res_log, s=7, alpha=0.3,
                color="#356B3F")
axes[1].axhline(0, color="black", lw=0.8)
axes[1].set(title="Log scale: formless band", xlabel="fitted (log)")
plt.show()

```

What just happened. We took logs of the price and of the two area columns, refitted, and re-ran the same test.

How to read the output. The Breusch–Pagan p-value moves from essentially 0 to about 0.18 — no evidence of heteroscedasticity left — and the right-hand panel is the formless band the left-hand one should have been. R^2 also improves, to about 0.84.

Why it matters. The fix was a change of *scale*, not of algorithm. When effects are multiplicative — a bigger house costs a certain *percentage* more, not a certain number of rupiah more — logs make them additive, which is what a linear model can represent.

TASK 6.1

Report the held-out R^2 of the **log-scale** model.

```

r2_log = None

checks.task("6.1", r2_log)

```

Hint. `model_log` is already fitted, and `.score(X, y)` returns R^2 . Use the held-out pair (`xte_1`, `yte_1`), not the training one — the value was printed two cells ago.

Careful with back-transformation. A coefficient of 0.08 on a log target means "about 8% higher", not "0.08 higher". And $\exp(\mathbb{E}[\log y])$ is the *geometric* mean, which sits below the arithmetic one.

```

coefs_log = pd.Series(model_log.coef_, index=Xlog.columns)
print("Log-model coefficients, read as elasticities / percentage effects:\n")
print(f"log(luas_bangunan) {coefs_log['luas_bangunan']:+.4f}"
      f" -> a 1% larger building implies "
      f"{coefs_log['luas_bangunan']:.2f}% higher price (elasticity)")
print(f"umur_bangunan {coefs_log['umur_bangunan']:+.4f}"
      f" -> each extra year is "
      f"{100*(np.exp(coefs_log['umur_bangunan'])-1):+.2f}% on price")
print(f"jarak_pusat_km {coefs_log['jarak_pusat_km']:+.4f}"
      f" -> each extra km is "
      f"{100*(np.exp(coefs_log['jarak_pusat_km'])-1):+.2f}% on price")

```

What just happened. We read the log-model's coefficients in the units people actually understand.

How to read the output. Two different readings, because two different kinds of predictor:

- `luas_bangunan` is *also* logged, so its coefficient is an **elasticity**: a 1% larger building implies about 0.64% higher price, at any size.
- `umur_bangunan` and `jarak_pusat_km` are not logged, so each extra unit multiplies the price by e^{β} — about −0.5% per year of age and −3% per kilometre from the centre.

Why it matters. On a log target, coefficients are percentages, not rupiah. Reporting 0.64 as "0.64 million per square metre" would be wrong by orders of magnitude.

The block everyone skips: residuals by predicted decile

A single RMSE averages over the whole range, so it hides *systematic* errors in one part of it. Split the predictions into ten groups and average the residual within each.

```

def residual_by_decile(actual, predicted):
    return (pd.Series(np.asarray(actual) - np.asarray(predicted))
            .groupby(pd.qcut(np.asarray(predicted), 10, labels=False))
            .mean())

pred_log_juta = np.exp(model_log.predict(Xte_1))    # back to millions of rupiah

print("mean residual by predicted decile (juta rupiah):\n")
print(pd.DataFrame({
    "raw-scale model": residual_by_decile(yte, pred).round(1),
    "log-scale model": residual_by_decile(np.exp(yte_1), pred_log_juta).round(1),
}).to_string())
print(f"\noverall mean residual, raw model : {(yte - pred).mean():+.2f}")
print(f"overall mean residual, log model : "
      f"{(np.exp(yte_1) - pred_log_juta).mean():+.2f}")

```

What just happened. We grouped the test houses into ten bins by *predicted* price and averaged the residual in each bin. Decile 0 is the cheapest tenth of predictions, decile 9 the most expensive.

How to read the output. The overall mean residual is -21.7 juta for the raw model — small against prices of several hundred million, and easy to wave through. The deciles show what it hides: the raw model under-predicts the cheapest tenth by $+66.9$, and then **every single remaining decile is negative**, by as much as -80.7 . A run of nine same-signed errors is not chance; the model systematically over-predicts everything above the bottom tenth. The log model's deciles change sign repeatedly, which is what noise looks like — though decile 5 ($+64.2$) is worth a look of its own.

Why it matters. This table takes three lines and finds problems RMSE cannot: a model that is right on average and wrong everywhere in particular. In the lecture's California housing example the same table reveals that the target is **censored** at $\$500,001$ — every more expensive house recorded at that one value — which no linear model can respect. That is a finding about the *data*, and it points to a different model, not a different hyperparameter.

3. Multicollinearity (DEMO)

Measured by the **variance inflation factor** $\text{VIF}_j = 1/(1 - R_j^2)$, where R_j^2 comes from regressing x_j on all the other predictors. The name is literal: the variance of $\hat{\beta}_j$ is inflated by exactly this factor.

```
def vif_table(df):
    out = {}
    cols = list(df.columns)
    for c in cols:
        others = [o for o in cols if o != c]
        r2 = LinearRegression().fit(df[others], df[c]).score(df[others], df[c])
        out[c] = 1 / (1 - r2) if r2 < 1 else np.inf
    return pd.Series(out).sort_values(ascending=False)

print("VIF on the house predictors:")
print(vif_table(rumah[feat_num]).round(2).to_string())
```

What just happened. For each predictor we asked how well the *other* predictors can reproduce it, and converted that into a variance inflation factor.

How to read the output. $\text{VIF} = 1$ means "unrelated to the others" — `jarak_pusat_km` and `umur_bangunan` are at 1.0. `luas_bangunan` and `kamar_tidur` sit around 5, which is mild: bigger houses have more bedrooms, so the columns overlap. The common rules of thumb are 5 (worth noting) and 10 (serious).

Why it matters. VIF is literal: the variance of that coefficient is multiplied by this number, so its standard error is multiplied by its square root. Nothing here is alarming yet — the next cell makes it alarming.

```
# Now inject a near-duplicate and watch what happens.
rumah_bad = rumah.copy()
rumah_bad["luas_bangunan_m2"] = (rumah_bad["luas_bangunan"]
                                + rng.normal(0, 0.5, len(rumah_bad)))

cols_bad = feat_num + ["luas_bangunan_m2"]
print("VIF with a near-duplicate column:")
print(vif_table(rumah_bad[cols_bad]).round(1).to_string())

m_clean = LinearRegression().fit(rumah[feat_num], rumah["harga"])
m_bad = LinearRegression().fit(rumah_bad[cols_bad], rumah_bad["harga"])

print(f"\ncoefficient on luas_bangunan, clean : {m_clean.coef_[0]:>12.3f}")
print(f"coefficient on luas_bangunan, with duplicate: "
      f"{m_bad.coef_[0]:>12.3f}")
print(f"coefficient on the duplicate          : "
      f"{m_bad.coef_[1]:>12.3f}")
print(f"their SUM                          : "
      f"{m_bad.coef_[0] + m_bad.coef_[1]:>12.3f}")
print(f"\nR^2 clean {m_clean.score(rumah[feat_num], rumah['harga']):.4f}    "
      f"R^2 with duplicate {m_bad.score(rumah_bad[cols_bad], rumah_bad['harga']):.4f}")
print("\nThe individual coefficients are meaningless; their SUM is stable;")
print("and the PREDICTIONS are unaffected. That is multicollinearity exactly.")
```

What just happened. We added a column that is `luas_bangunan` plus a tiny amount of noise — the same information twice — and refitted.

How to read the output. Three things at once:

- VIF for the two twins explodes into the thousands.
- The coefficient on `luas_bangunan` flips from about +6.3 to about −5.9, while the duplicate takes +12.2. Both are nonsense on their own; their **sum** (≈ 6.3) is the original, stable answer.
- R^2 is unchanged to four decimals: the **predictions did not move**.

Why it matters. Collinearity is a problem of *interpretation*, not of prediction. If you only need forecasts, ignore it. If anyone will read a coefficient — a regulator, a colleague, a policy — it is fatal, and no adjustment to standard errors repairs it. Remedies: drop one twin, combine them, or use ridge (§4), which shares the effect between them instead of flipping signs.

4. Regularisation (DEMO)

Expected error decomposes as $\text{bias}^2 + \text{variance} + \sigma^2_{\text{irreducible}}$.

Regularisation buys a reduction in variance at the price of a little bias.

- **Ridge** (ℓ_2): shrinks, never to exactly zero; tames collinearity.
- **Lasso** (ℓ_1): sets coefficients **exactly** to zero — it selects.
- **Elastic net**: both; keeps correlated groups together.

This reproduces Worked Example 6.2 of the book, seed and all, so your numbers match the book's and the lecture slide's exactly.

```

from sklearn.linear_model import RidgeCV, LassoCV, ElasticNetCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# A hard case: p large relative to n, and the truth is sparse.
# Seeded separately (42, as in the book) so this section stands on its own.
rng_reg = np.random.default_rng(42)
n_s, p_s = 200, 60
Xs = rng_reg.normal(size=(n_s, p_s))
Xs[:, 1] = Xs[:, 0] + rng_reg.normal(0, 0.1, n_s) # near-duplicates of feature 0
Xs[:, 2] = Xs[:, 0] + rng_reg.normal(0, 0.1, n_s)
beta_true = np.zeros(p_s)
beta_true[:5] = [3.0, 0.0, 0.0, -2.0, 1.5]
ys = Xs @ beta_true + rng_reg.normal(0, 2.0, n_s)

Xs_tr, Xs_te, ys_tr, ys_te = train_test_split(Xs, ys, test_size=0.3,
                                              random_state=0)

alphas = np.logspace(-3, 3, 60)

models = {
    "OLS": make_pipeline(StandardScaler(), LinearRegression()),
    "Ridge": make_pipeline(StandardScaler(), RidgeCV(alphas=alphas)),
    "Lasso": make_pipeline(StandardScaler(),
                          LassoCV(alphas=alphas, max_iter=50_000,
                                random_state=0)),
    "ElasticNet": make_pipeline(StandardScaler(),
                               ElasticNetCV(l1_ratio=[.1, .5, .7, .9, .95, 1],
                                             alphas=alphas, max_iter=50_000,
                                             random_state=0)),
}

rows = []
for name, m in models.items():
    m.fit(Xs_tr, ys_tr)
    coef = m[-1].coef_
    rows.append({"model": name,
                "test_RMSE": mean_squared_error(ys_te, m.predict(Xs_te)) ** 0.5,
                "non_zero": int((np.abs(coef) > 1e-8).sum())})
print(pd.DataFrame(rows).to_string(index=False, float_format=lambda v: f"{v:.3f}"))
print(f"\nirreducible noise floor: sigma = 2.0")
print("Only 3 of the 60 coefficients are genuinely non-zero.")

```

What just happened. 200 rows, 60 predictors, but only three of them actually affect the target (and two more are near-copies of the first). Four models, same data, same split.

How to read the output. Read RMSE against the noise floor of 2.0, which no model can beat:

- **OLS 2.182** — the worst, because it spends 60 coefficients fitting noise.
- **Ridge 2.049** and **Lasso 2.068** — both better, and *the difference between them is well inside the noise* of a 60-row test set.
- But ridge keeps **60** non-zero coefficients and lasso keeps **7**.

Why it matters. Regularised models rarely win by a lot of accuracy. They win by reaching the *same* accuracy with far less complexity. If you need a prediction, either will do; if you need an explanation, or a model somebody must maintain and audit, seven coefficients is a different kind of object from sixty.

Choosing λ by cross-validation, and the one-standard-error rule

Training error always prefers $\lambda = 0$, so the penalty must be chosen on held-out data. **LassoCV** did that above; here is the curve it used, plus the convention that goes one step further.

```

from sklearn.model_selection import cross_val_score, KFold
from sklearn.linear_model import Lasso

Z_tr = StandardScaler().fit_transform(Xs_tr)
alpha_grid = np.logspace(-3, 1, 40)
cv5 = KFold(5, shuffle=True, random_state=0)

means, ses = [], []
for a in alpha_grid:
    sc = -cross_val_score(Lasso(alpha=a, max_iter=50_000), Z_tr, ys_tr,
                          cv=cv5, scoring="neg_mean_squared_error")
    means.append(sc.mean())
    ses.append(sc.std() / np.sqrt(5))
means, ses = np.array(means), np.array(ses)

i_min = int(np.argmin(means))
i_1se = int(np.max(np.where(means <= means[i_min] + ses[i_min])[0]))
nz = lambda a: int((np.abs(Lasso(alpha=a, max_iter=50_000)
                              .fit(Z_tr, ys_tr).coef_) > 1e-8).sum())

print(f"minimum CV error : alpha = {alpha_grid[i_min]:.4f}    "
      f"MSE = {means[i_min]:.3f}    non-zero = {nz(alpha_grid[i_min])}")
print(f"one-SE rule      : alpha = {alpha_grid[i_1se]:.4f}    "
      f"MSE = {means[i_1se]:.3f}    non-zero = {nz(alpha_grid[i_1se])}")
print(f"(the truth has 3 non-zero coefficients)")

fig, ax = plt.subplots(figsize=(6.5, 3.6))
ax.errorbar(alpha_grid, means, yerr=ses, fmt="o-", ms=3, lw=1,
            color="#173F5F", ecolor="#B8C4D0", capsize=2)
ax.axvline(alpha_grid[i_min], color="#356B3F", ls="--",
            label=f"minimum ({nz(alpha_grid[i_min])} coefficients)")
ax.axvline(alpha_grid[i_1se], color="#9C3535", ls=":",
            label=f"one-SE rule ({nz(alpha_grid[i_1se])} coefficients)")
ax.axhline(means[i_min] + ses[i_min], color="grey", lw=0.8)
ax.set(xscale="log", xlabel=r"penalty $\alpha$", ylabel="CV mean squared error",
        title="The CV curve, and both conventions on it")
ax.legend(fontsize=8, frameon=False)
plt.show()

```

What just happened. For each candidate penalty we ran 5-fold CV on the training rows and recorded the mean error and its standard error (the bars).

How to read the output. The curve is U-shaped: too little penalty overfits, too much underfits. Two ways to read it:

- **The minimum** — the lowest point, here about 3.9, keeping 7 coefficients.
- **The one-standard-error rule** — the *largest* penalty whose error is still within one standard error of that minimum (the grey line). Here it keeps **4** coefficients at an error of about 4.1.

The truth has 3. The one-SE model is the closer description, for a cost that is statistically indistinguishable from the best.

Why it matters. The minimum is the best guess for prediction; the one-SE rule prefers the simplest model the data cannot distinguish from it. Choosing between them is a decision about what the model is for — and both are defensible, which is why you should say which one you used.

The coefficient path

Watch lasso zero things out as the penalty grows.

```

from sklearn.linear_model import lasso_path, ridge_regression

Xs_std = StandardScaler().fit_transform(Xs)
alpha_path, coefs_path, _ = lasso_path(Xs_std, ys, alphas=np.logspace(-2, 1, 60))
ridge_path = np.array([ridge_regression(Xs_std, ys, alpha=a)
                       for a in np.logspace(-2, 4, 60)]).T

fig, axes = plt.subplots(1, 2, figsize=(11, 3.6))
for i in range(p_s):
    hot = i in (0, 3, 4)
    axes[0].plot(alpha_path, coefs_path[i],
                  color="#9C3535" if hot else "#B8C4D0",
                  lw=1.6 if hot else 0.6, zorder=3 if hot else 1)
    axes[1].plot(np.logspace(-2, 4, 60), ridge_path[i],
                  color="#9C3535" if hot else "#B8C4D0",
                  lw=1.6 if hot else 0.6, zorder=3 if hot else 1)
for ax, title in zip(axes, ["Lasso: coefficients hit exactly zero",
                             "Ridge: shrinks, never reaches zero"]):
    ax.set(xscale="log", xlabel=r"penalty $\lambda$", ylabel="coefficient",
           title=title)
    ax.axhline(0, color="black", lw=0.6)
fig.suptitle("Red = the three genuinely non-zero coefficients", y=1.05,
             fontsize=9, color="#9C3535")
plt.show()

```

How to read the figure. Each thin line is one of the 60 coefficients as the penalty grows from left to right; the three red lines are the real signals.

Left (lasso). Lines hit zero and *stay* there, one after another. A vertical slice through this panel is a **selected model** — the survivors at that penalty. The red lines survive longest, which is the behaviour you want.

Right (ridge). Everything shrinks smoothly towards zero and nothing ever arrives. A vertical slice gives you sixty small numbers, never a short list.

Why it matters. This is the whole difference between the two penalties, in one picture: ℓ_1 has a corner at zero and ℓ_2 does not.

TASK 6.2

Which penalty performs **variable selection** by setting coefficients exactly to zero?

```

penalti = None          # "ridge", "lasso" or "elasticnet"

checks.task("6.2", penalti)

```

Hint. Look at the **non_zero** column of the table above, or at which panel of the figure has lines that actually reach zero.

5. Logistic regression (DEMO)

Linear regression is unsuitable for a binary target: it predicts outside $[0,1]$ and its error structure is wrong. Model the **log-odds** instead:

$$\log \frac{p}{1-p} = \mathbf{x}^T \boldsymbol{\beta} \quad \Longleftrightarrow \quad p = \frac{1}{1 + e^{-\mathbf{x}^T \boldsymbol{\beta}}}$$

```

fig, ax = plt.subplots(figsize=(5.5, 3.2))
z = np.linspace(-6, 6, 300)
ax.plot(z, 1 / (1 + np.exp(-z)), color="#173F5F", lw=2)
ax.axhline(0.5, color="grey", ls=":", lw=1)
ax.axvline(0, color="grey", ls=":", lw=1)
ax.set(xlabel=r"log-odds  $\mathbf{x}^{\top}\beta$", ylabel="probability",
       title="The logistic (sigmoid) function")
ax.annotate("nearly linear here", xy=(0, 0.5), xytext=(2.2, 0.32),
           arrowprops=dict(arrowstyle="->", color="#9C3535"), fontsize=8,
           color="#9C3535")
ax.annotate("very flat here", xy=(-4.5, 0.011), xytext=(-5.8, 0.28),
           arrowprops=dict(arrowstyle="->", color="#9C3535"), fontsize=8,
           color="#9C3535")
plt.show()

```

How to read the figure. The sigmoid converts any number on the horizontal axis — the log-odds, which can be anything from $-\infty$ to $+\infty$ — into a probability between 0 and 1. It is steepest at 0 (probability 0.5) and almost flat in the tails.

Why it matters. That changing steepness is the reason the *same* change in log-odds moves the probability a lot in the middle and hardly at all near 0 or 1 — which is the point of the odds-ratio cell below.

```

from sklearn.linear_model import LogisticRegression
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.metrics import (accuracy_score, roc_auc_score, log_loss,
                             confusion_matrix, roc_curve)

credit = pd.read_csv(DATA / "credit.csv")
yc = (credit["default"] == "yes").astype(int)
Xc = credit.drop(columns=["default"]).copy()
Xc["dti"] = Xc["debt"] / Xc["income"].clip(lower=1)
Xc["pti"] = Xc["monthly_payment"] / (Xc["income"] / 12).clip(lower=1)
Xc["util"] = Xc["balance"] / Xc["credit_limit"].clip(lower=1)

num_c = Xc.select_dtypes("number").columns.tolist()
cat_c = Xc.select_dtypes("object").columns.tolist()

prep = ColumnTransformer([
    ("n", Pipeline([("i", SimpleImputer(strategy="median")),
                    ("s", StandardScaler())]), num_c),
    ("c", Pipeline([("i", SimpleImputer(strategy="constant",
                                         fill_value="Unknown")),
                    ("o", OneHotEncoder(handle_unknown="ignore",
                                         sparse_output=False))]), cat_c),
], verbose_feature_names_out=False)

clf = Pipeline([("prep", prep),
                ("lr", LogisticRegression(max_iter=5000, C=1.0))])

Xc_tr, Xc_te, yc_tr, yc_te = train_test_split(Xc, yc, test_size=0.25,
                                              stratify=yc, random_state=42)

clf.fit(Xc_tr, yc_tr)
proba = clf.predict_proba(Xc_te)[: , 1]
pred_c = (proba >= 0.5).astype(int)

print(f"accuracy : {accuracy_score(yc_te, pred_c):.4f}")
print(f"ROC-AUC : {roc_auc_score(yc_te, proba):.4f}")
print(f"log loss : {log_loss(yc_te, proba):.4f}")
print(f"\nbase rate (always predict 'no default'): "
      f"{1 - yc_te.mean():.4f} <- compare with accuracy!")
print("\nconfusion matrix (rows=true, cols=predicted):")
print(confusion_matrix(yc_te, pred_c))

```

What just happened. A full pipeline — impute, scale, one-hot, fit logistic regression — trained on 75% of the applications and scored on the rest, with the three week-5 ratios included.

How to read the confusion matrix. Rows are the truth, columns the prediction. The top-left is correctly-approved, bottom-right correctly caught defaulters. The bottom-**left** cell is the expensive one: applicants who defaulted and the model let through.

The trap in the accuracy. 0.837 looks respectable until you see the base rate: predicting "no default" for *everyone* scores 0.800. The model buys under 4 points, and the confusion matrix shows why — at a threshold of 0.5 it catches only a third of the defaulters. Meanwhile its ROC-AUC of 0.82 says it ranks applicants well.

Why it matters. Accuracy depends entirely on the 0.5 cut; AUC integrates over every possible cut. The ranking is good and the *operating point* is bad, which is a fixable problem — §6.

Coefficients as odds ratios

```
names = clf.named_steps["prep"].get_feature_names_out()
coefs = pd.Series(clf.named_steps["lr"].coef_[0], index=names)
odds = np.exp(coefs)

top = pd.DataFrame({"coef": coefs, "odds_ratio": odds})
top = top.reindex(top["coef"].abs().sort_values(ascending=False).index).head(10)
print("Largest effects (features standardised, so these are comparable):\n")
print(top.round(4).to_string())
```

What just happened. We exponentiated each coefficient. On the log-odds scale effects add; exponentiating turns them into multipliers of the **odds**.

How to read the output. An odds ratio above 1 raises the odds of default, below 1 lowers them. **pti** at about 2.2 means: one standard deviation more payment-to-income multiplies the odds of default by 2.2, holding the others fixed. **income** at about 0.62 lowers them by a third.

Because every numeric feature was standardised inside the pipeline, these are comparable with each other — each answers the same "per one standard deviation" question. On raw scales they would not be: a coefficient per rupiah and one per year cannot be ranked.

Why it matters. The top two are **pti** and **dti**, the engineered ratios from week 5. And note the ranking is of *association*, not of causation — and collinear predictors (§3) split a shared effect, so read them as a group.

e^{β_j} is the multiplicative change in the **odds** for a one-SD increase — *not* in the probability. The distinction matters enormously:

```
beta = 0.693          # exactly doubles the odds
for p0 in (0.05, 0.10, 0.20, 0.50, 0.80):
    odds0 = p0 / (1 - p0)
    p1 = (odds0 * np.exp(beta)) / (1 + odds0 * np.exp(beta))
    print(f"baseline p = {p0:.2f} -> p = {p1:.4f} "
          f"(change {100*(p1-p0):+.2f} percentage points)")

print("\nThe SAME doubling of the odds moves the probability by 4.5 points at")
print("p=0.05, by 8.2 points at p=0.10, and by 16.7 points at p=0.50.")
print("Never say 'the probability doubles'.")
```

What just happened. One fixed coefficient ($\beta = 0.693$, so $e^{\beta} = 2$, exactly doubling the odds) applied at five different starting probabilities.

How to read the output. The same doubling of the odds produces a completely different change in probability: +4.5 points at a baseline of 0.05, +8.2 at 0.10, +16.7 at 0.50, and only +8.9 at 0.80. Largest in the middle, small at both extremes — the shape of the sigmoid, again.

Why "the probability doubles" is always wrong. At $p = 0.80$ it would require a probability of 1.6. Odds double; probabilities cannot.

Why it matters. Report the effect as a probability *at a stated baseline*: "for an applicant like this one, at 10%, it rises to 18%." That is the sentence a stakeholder can repeat without misleading anyone.

Two implementation facts that surprise people.

1. **LogisticRegression is regularised by default.** Its `C` is the *inverse* penalty strength, so small `C` means strong shrinkage, and the default `C=1.0` is already a penalised fit. `statsmodels`' `Logit` is unpenalised, so the two disagree until you set `penalty=None`. If you are reporting coefficients for inference, say which you used.
2. **For imbalance, try `class_weight="balanced"` first.** It reweights the loss by inverse class frequency — no resampling, no extra variance, one argument.

6. Choosing the threshold from costs (DEMO)

A classifier outputs a probability. Converting it to a decision needs a threshold, and **0.5 is a default, not an answer**.

Acting positive costs $(1-p)C_{\text{FP}}$; acting negative costs pC_{FN} . Acting positive is preferable when $p > \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}}$

```
def total_cost(y_true, proba, thr, c_fp, c_fn):
    tn, fp, fn, tp = confusion_matrix(y_true,
                                      (proba >= thr).astype(int)).ravel()
    return fp * c_fp + fn * c_fn

C_FP, C_FN = 1.0, 10.0      # missing a default costs 10x a needless rejection
theory = C_FP / (C_FP + C_FN)

grid = np.linspace(0.02, 0.98, 97)
costs = [total_cost(y_true, proba, t, C_FP, C_FN) for t in grid]
best = grid[int(np.argmin(costs))]

print(f"cost ratio C_FN/C_FP = {C_FN/C_FP:.0f}")
print(f"theoretical threshold = {theory:.4f}")
print(f"empirical minimum      = {best:.4f}")
print(f"\ncost at threshold 0.5 : {total_cost(y_true, proba, 0.5, C_FP, C_FN):.0f}")
print(f"cost at threshold {best:.2f} : "
      f"{total_cost(y_true, proba, best, C_FP, C_FN):.0f}")
print(f"saving: "
      f"{1 - total_cost(y_true, proba, best, C_FP, C_FN) / total_cost(y_true, proba, 0.5, C_FP, C_FN):.1%}")
```

What just happened. We priced every possible threshold. Each false positive (rejecting a good applicant) costs 1; each false negative (missing a defaulter) costs 10. Then we found the cheapest threshold.

How to read the output. The formula says $1/(1+10) = 0.0909$; the empirical minimum on this test set is 0.08 — the same answer up to the granularity of the grid and the noise of a finite sample. Moving from the default 0.5 to that threshold cuts total cost by more than half.

Why it matters. Nothing about the model changed — same coefficients, same probabilities, same AUC. Only the cut-off moved. A threshold is a *business* decision, and leaving it at 0.5 silently asserts that both errors cost the same.

```

fig, axes = plt.subplots(1, 2, figsize=(11, 3.6))

axes[0].plot(grid, costs, color="#173F5F")
axes[0].axvline(best, color="#356B3F", ls="--",
                label=f"empirical min {best:.2f}")
axes[0].axvline(theory, color="#9C3535", ls=":",
                label=f"theory {theory:.2f}")
axes[0].axvline(0.5, color="grey", ls="-.", label="default 0.5")
axes[0].set(xlabel="threshold", ylabel="total cost",
            title=f"Cost curve (C_FN = {C_FN:.0f} x C_FP)")
axes[0].legend(fontsize=8, frameon=False)

fpr, tpr, _ = roc_curve(y_c_te, proba)
axes[1].plot(fpr, tpr, color="#2E6F9E",
             label=f"AUC = {roc_auc_score(y_c_te, proba):.3f}")
axes[1].plot([0, 1], [0, 1], ls=":", color="grey")
axes[1].set(xlabel="false positive rate", ylabel="true positive rate",
            title="ROC curve")
axes[1].legend(fontsize=8, frameon=False)
plt.show()

```

How to read the figure. Left, total cost against threshold: a clear minimum far to the left of the default 0.5, with the theoretical value sitting right beside the empirical one. The curve is flat-bottomed, so any threshold in that region is about as good — useful to know when the costs themselves are only estimates.

Right, the ROC curve: every threshold at once, as a trade between catching defaulters (vertical) and wrongly rejecting good applicants (horizontal). The diagonal is random guessing; the area under the curve is the AUC. Choosing a threshold is choosing a *point* on this curve.

TASK 6.3

If a false negative costs **10×** a false positive, what is the theoretically optimal threshold?

```

ambang = None

checks.task("6.3", ambang)

```

Hint. Use the formula, not the empirical minimum: $C_{\text{FP}} / (C_{\text{FP}} + C_{\text{FN}})$ with $C_{\text{FP}} = 1$ and $C_{\text{FN}} = 10$.

This calculation requires **calibrated** probabilities: among instances predicted at 0.3, roughly 30% must actually be positive. Logistic regression is usually well calibrated by construction; tree ensembles and SVMs are not, and need `CalibratedClassifierCV` first.

```
# A quick calibration check: bin the predictions and compare with reality.
bins = pd.qcut(proba, 10, duplicates="drop")
cal = pd.DataFrame({"predicted": proba, "actual": yc_te.to_numpy(),
                    "bin": bins}).groupby("bin", observed=True).agg(
    mean_predicted=("predicted", "mean"),
    observed_rate=("actual", "mean"),
    n=("actual", "size"))
print(cal.round(4).to_string())

fig, ax = plt.subplots(figsize=(4.2, 4))
ax.plot([0, cal["mean_predicted"].max()], [0, cal["mean_predicted"].max()],
        ls=":", color="grey", label="perfect calibration")
ax.plot(cal["mean_predicted"], cal["observed_rate"], marker="o", ms=5,
        color="#2E6F9E", label="logistic regression")
ax.set(xlabel="mean predicted probability", ylabel="observed frequency",
        title="Calibration plot")
ax.legend(fontsize=8, frameon=False)
plt.show()
```

What just happened. We sorted the test applicants into ten groups by predicted probability and compared each group's average prediction with what actually happened in it.

How to read the output. The two columns track each other closely — the group predicted at 0.05 defaults about 5% of the time, the group predicted at 0.71 about 69%. On the plot the points sit near the diagonal, wobbling by a few points either way on 100 cases per bin.

Why it matters. This is what makes the cost calculation legitimate. A threshold of 0.09 means "act when the risk exceeds 9%", which is only a sensible instruction if 9% means 9%. Run this check before trusting any threshold — especially with a model that is *not* logistic regression.

7. Take-home (assessed)

A. Derive it. Derive $\hat{\beta}_1 = \frac{\text{Cov}(x,y)}{\text{Var}(x)}$ from the two normal equations, showing every step. Then show the residuals sum to zero whenever an intercept is included, and explain the **geometric** meaning.

B. Diagnose four violations. For each, state the consequence for the coefficients, for their standard errors, and the appropriate remedy: (a) heteroscedastic errors; (b) an omitted variable correlated with an included one; (c) VIF of 25 on two predictors; (d) heavy-tailed residuals at $n = 5000$. Which one is the *serious* case, and why?

C. Sparsity and lasso. Reproduce section 4, then change the number of truly relevant predictors from 3 to 40 and rerun. Report which method now wins and explain the result in terms of the sparsity assumption underlying lasso.

D. Odds vs probability. A logistic model of default on income (in millions of rupiah) gives $\beta = -0.35$. Compute the odds ratio and interpret it in one sentence. Then compute the change in default *probability* for a one-million increase for applicants at baseline (a) 0.05 and (b) 0.40, and explain why the two answers differ.

E. Cost-sensitive deployment. A fraud model's costs are: investigating a legitimate transaction Rp 50,000; missing a fraudulent one Rp 4,000,000. Compute the optimal threshold, then state what additional property the model's outputs must have for that threshold to be valid — and how you would check it.

Next week: the classification toolkit, compared fairly.